SOFTWARE DESIGN
& ANALYSIS

Complete Study Guide

Lectures 01–08 · Process Models · UML · Requirements · Domain Model

Includes: Waterfall · V-Model · Rapid Prototyping · Incremental · Spiral

SECTION 1: FOUNDATIONS OF SOFTWARE ENGINEERING

1.1 What is Software?

Software is a set of instructions (programs) that tells hardware what to do. It acts as a bridge between the user and the physical machine. Without software, hardware is useless — it has no idea what task to perform.

> **Simple Definition:**
> Software = Instructions + Data that enable a computer to perform specific tasks for users.

1.2 What is Software Engineering?

Software Engineering is a systematic, disciplined, and quantifiable approach to the development, operation, and maintenance of software. The key word is SYSTEMATIC — without a system and process, software development turns into chaos.

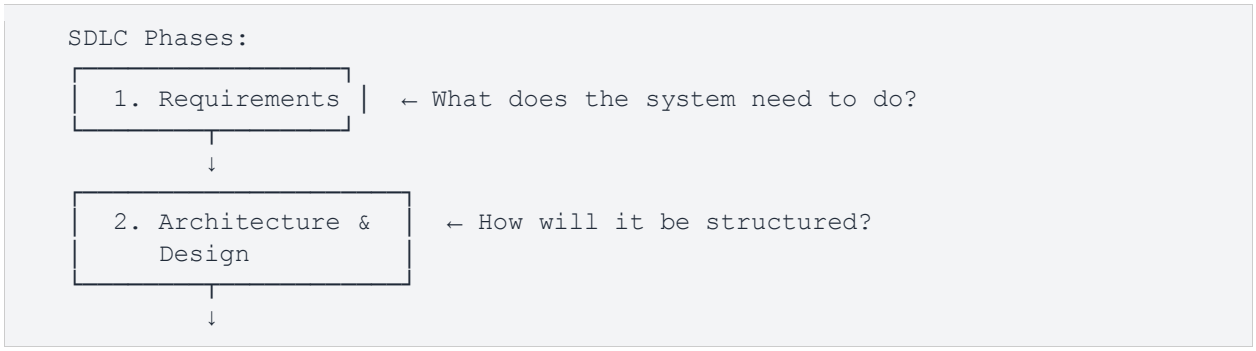
> **Key Point: Software Engineering prevents chaos. Without it, teams build the wrong thing, go over budget, miss deadlines, and deliver buggy software.**

Why is a systematic approach needed?

- Requirements change frequently — a structured approach handles changes properly
- Multiple developers working together need clear guidelines and standards
- Software failures can be costly or even life-threatening (medical, aviation systems)
- It ensures the final product is reliable, maintainable, and scalable

1.3 SDLC — Software Development Life Cycle

The SDLC is the overall process that every software project follows. It defines the phases from the very beginning of an idea to the retirement of the software.



1.4 Project vs Product

	Project	Product
Client	Specific known client	No specific audience — built for everyone
Requirements	Easier to gather — client is present	Much harder — must predict what users want
Example	Build an ERP for ABC Company	Build Microsoft Word / Instagram
Risk	Lower — direct feedback possible	Higher — market may not want it

1.5 Stakeholders

A stakeholder is ANYONE who is affected by, or who affects, the software. This is broader than just the client.

- Client / Customer — the person who commissioned the software
- End Users — people who will actually use the system daily
- Developers — they build it (they are direct affectees too!)
- Managers — oversee timelines and budgets
- Regulatory bodies — may impose legal requirements

Important: Wants can come from stakeholders too — e.g. a developer adding a colour scheme they personally prefer, without client approval. This leads to Goldplating.

1.6 Goldplating & Scope Creep

Goldplating

Goldplating occurs when a developer adds extra features, functionalities, or improvements that the client NEVER requested. It seems helpful but is actually harmful.

> **Example of Goldplating:**
> A client asked for a simple inventory system. The developer added AI demand-forecasting, a mobile app, and multi-language support — all without approval. The project went over budget by 40% and was delivered 3 months late.

Consequences of Goldplating:

- Cost overrun — extra features consume budget not planned for
- Time delays — team spends time on unwanted features instead of core ones
- Bugs in core features — resources diverted from what actually matters
- Client dissatisfaction — the product doesn't match what they envisioned

Scope Creep

Scope Creep happens when the project's target/boundary gradually drifts from the original agreed scope — usually because the client keeps adding new requirements mid-project.

> **Example of Scope Creep:**
> A team was hired to build an e-commerce website. Midway through, the client kept adding: 'Add a blog. Add a loyalty program. Support 5 languages. Add a mobile app.' The timeline kept extending and original features were never fully completed.

Who is responsible? While the CLIENT is the one adding requests, the REQUIREMENT ENGINEER must control it — guide the client, stick to agreed requirements, and evaluate feasibility of new requests before accepting them.

	Goldplating	Scope Creep
Who causes it?	Developer adds extras	Client keeps requesting more
Root cause	Developer over-enthusiasm	Lack of change control process
Fix	Stick strictly to requirements	RE must control and evaluate new requests
Both result in	Cost overrun, delays, chaos	Cost overrun, delays, chaos

SECTION 2: REQUIREMENTS ENGINEERING

2.1 Why Requirements Matter

The 70% Rule: 70% of all software failures are due to wrong work in the requirements phase — bad analysis or requirements not understood correctly.

2.2 The Requirement Engineer — 3 Phases

Requirements work is so critical it requires a full-time professional — the Requirement Engineer (RE) or Business Analyst. Their work is split into 3 phases:

Phase	What Happens	Key Output
1. Elicitation	Gathering requirements through interviews, discussions, workshops, observation. Takes consensus from ALL stakeholders.	List of stakeholder needs and wants
2. Analysis	Verifying requirements are understood correctly. Checking completeness, consistency, and feasibility. Distinguishing Needs from Wants.	Refined, validated requirements
3. Documentation	Writing requirements formally (SRS) or informally. Creates the agreement between client and development team.	SRS document or informal specs

2.3 Needs vs Wants

During elicitation, clients often mix what they NEED (real requirements) with what they WANT (nice-to-haves). The RE must separate these clearly.

- Needs → The REAL requirements. These must be implemented. Without them the system fails its purpose.
- Wants → Things the client would like but are not essential. Can become scope creep if not managed.

Example:

Client says: 'I need a login system and I also want the dashboard to use my company colours and have animations on every page.'

NEED = Login system (core functionality). WANT = animations (nice to have, not required).

2.4 Functional Requirements (FR)

Functional Requirements define WHAT the system does — the specific behaviours, functions, and features the system must provide.

- FR = Features and functions that users interact with directly
- They are directly mapped to Use Cases in UML
- Examples: Login, Search, Book Ticket, Cancel Order, View History, Generate Report

> **FR Examples for an Online Store:**
>
> FR1: The system shall allow customers to search for products by name, category, or price range.
> FR2: The system shall allow registered users to add products to a shopping cart.
> FR3: The system shall send an email confirmation after a successful order is placed.
> FR4: Admin shall be able to add, update, and delete product listings.

2.5 Non-Functional Requirements (NFR)

Non-Functional Requirements define HOW WELL the system performs. They are quality attributes — not features, but properties of the system.

NFR Type	Explanation & Example
Performance	How fast the system responds. 'Search results must load within 2 seconds for 5,000 users.'
Security	Protection of data. 'Passwords must be stored with bcrypt hashing. All data via HTTPS.'
Reliability	How often it works without failure. '99.9% uptime guaranteed.'
Robustness	Ability to RECOVER after failure. 'System must recover within 5 seconds of any crash.'
Usability	How easy it is to use. 'A new user must complete checkout without training in under 5 minutes.'
Scalability	Handle growth. 'Must support 1 million users without performance degradation.'
Compatibility	Works across environments. 'Must run on Chrome, Firefox, Safari, and Edge.'

NFR → FR Rule: Sometimes NFR becomes FR. Example: Security is normally NFR — but if the app handles cash transactions, encryption becomes a FUNCTIONAL requirement (it must be coded in as a feature).

2.6 SRS — Software Requirements Specification

The SRS is the formal document that captures ALL agreed requirements. It is the contract between the client and the development team.

- Formal SRS: Has a specific structured format — used for large, complex projects

- Informal documentation: No fixed format — used for small, quick projects

Why is it critical? Without a signed SRS, both parties may have different understandings of what was agreed. This is why the 70% failure rule exists — verbal agreements are not enough.

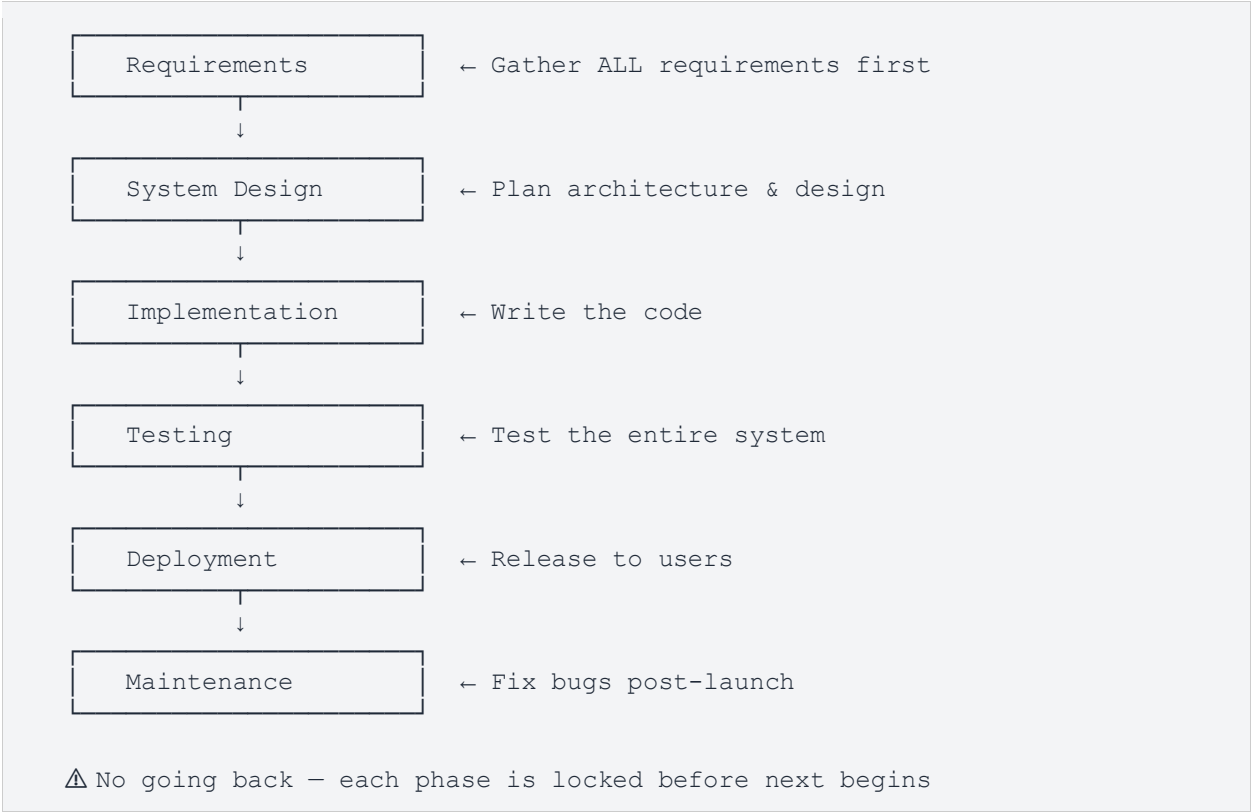
SECTION 3: SOFTWARE PROCESS MODELS

A Software Process Model is a framework that defines the ORDER and STRUCTURE of the phases in software development. It tells the team: how to plan, design, build, and test the software. Choosing the right model is critical — the wrong model can lead to project failure.

① Waterfall Model

What is it?

The Waterfall Model is the oldest and most traditional SDLC model. Phases flow strictly DOWNWARD like a waterfall — one phase must be fully completed before the next begins. There is no going back.



✓ Advantages	✗ Disadvantages
Simple to understand and manage	Extremely inflexible — changes are expensive
Clear milestones — easy to track progress	Testing happens LATE — bugs found after all code written
Heavy documentation at each phase	Customer sees product only at the very END

Works well for fixed, well-understood projects	Real-world requirements rarely stay fixed
Easy to divide work among teams	High risk — problems discovered too late

✓When to Use Waterfall:

Requirements are completely clear, fixed, and well-documented from the start
Short, small, well-defined projects with no expected changes
Government or military contracts where strict documentation is required
Projects with regulatory compliance requirements
Example: Building payroll software for a company with legally defined pay rules

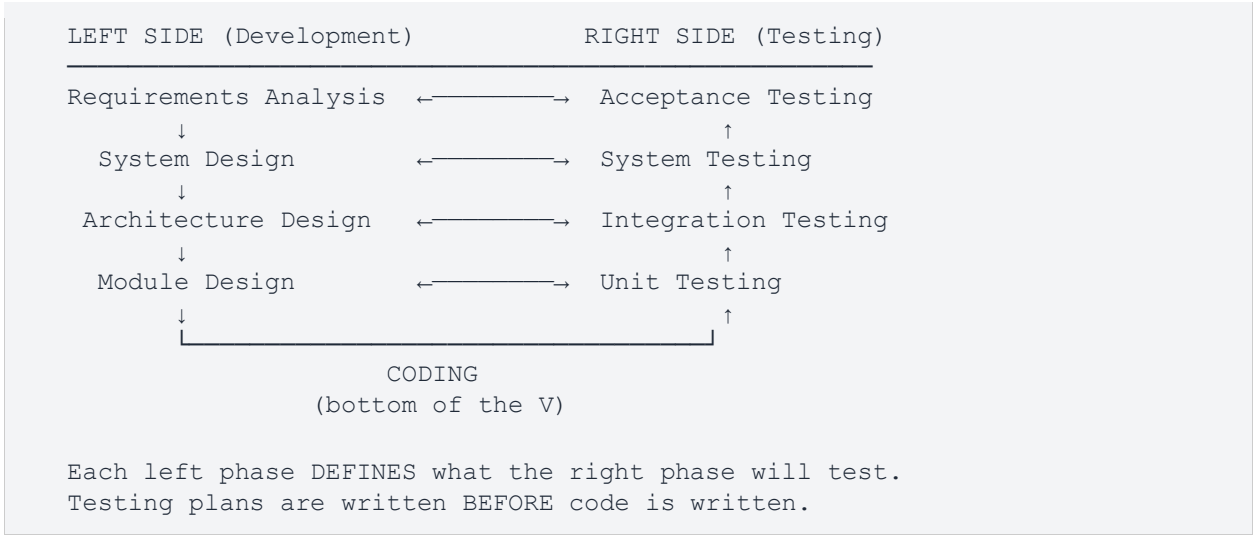
✗When NOT to Use Waterfall:

When requirements are unclear or likely to change during development
Long projects where technology or business needs may shift
When the client needs to see progress or provide feedback regularly
Any modern, user-facing application — requirements always evolve

② V-Model (Verification & Validation)

What is it?

The V-Model is an extension of Waterfall shaped like the letter 'V'. The key innovation: every DEVELOPMENT phase on the left has a corresponding TESTING phase planned simultaneously on the right. Testing is not an afterthought — it is designed in parallel.



✓ Advantages	✗ Disadvantages

Testing planned EARLY — before any code written	Still sequential — inflexible to requirement changes
Defects found earlier = cheaper to fix	Expensive to go back and fix errors
Clear entry/exit criteria for each phase	No working prototype shown to client until late
Works well for safety-critical systems	Not suitable for long projects with evolving needs
High quality output with thorough verification	Requires very detailed requirements upfront

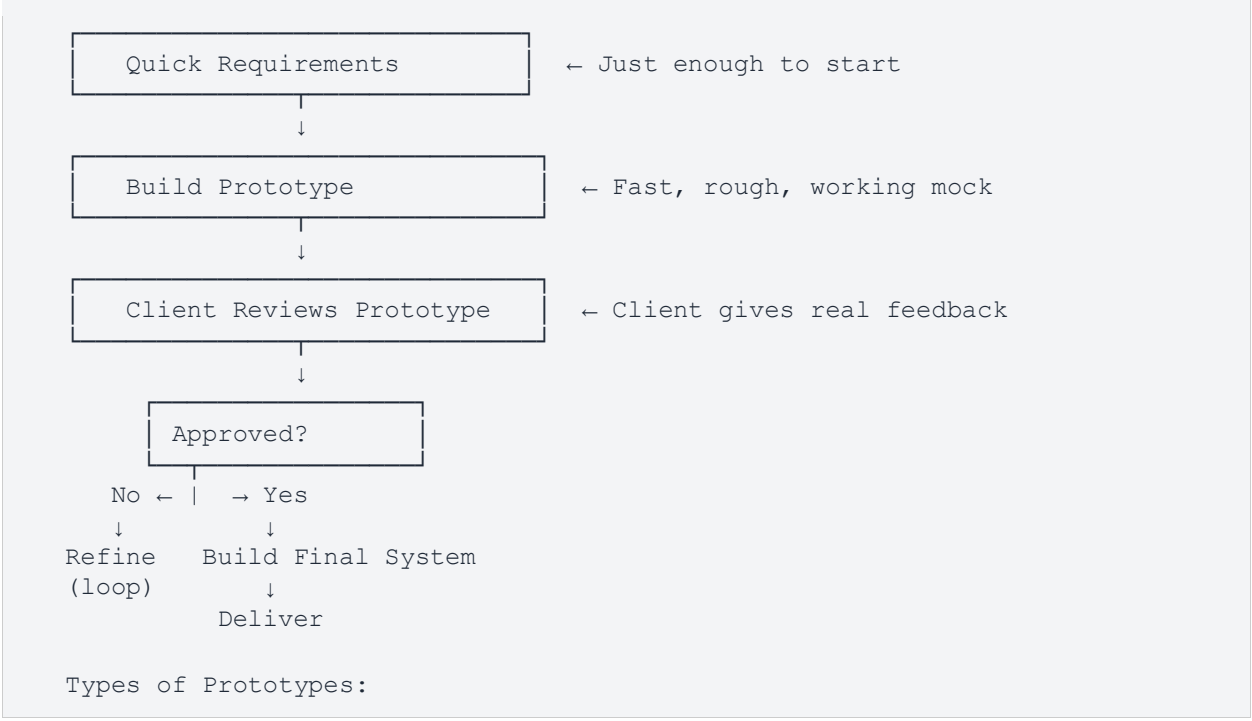
✓When to Use V-Model:

Safety-critical systems where every requirement must be formally verified
Medical device software (FDA compliance requires verification at each stage)
Aerospace and aviation systems
Defense and military software
Projects where quality assurance is the highest priority
Example: Software controlling a hospital's drug dispensing machine

③ Rapid Prototyping Model

What is it?

Instead of building the full system right away, the team quickly creates a rough working prototype — a simplified mock-up — and shows it to the client EARLY to gather feedback. The prototype is then refined until the client is satisfied, then the real system is built.



```
- Throwaway Prototype: Built for feedback only, then discarded
- Evolutionary Prototype: Refined into the final system
```

✓ Advantages	✗ Disadvantages
Client sees working software EARLY	Client may think prototype IS the final product
Misunderstandings caught before full development	Developers may carry over shortcuts from prototype
Reduces risk of building the wrong product	Can lead to scope creep if client keeps requesting changes
Great for UI-heavy systems where look & feel matters	Incomplete analysis may result in poor architecture
Encourages active client participation	Hard to manage if feedback cycles go on indefinitely

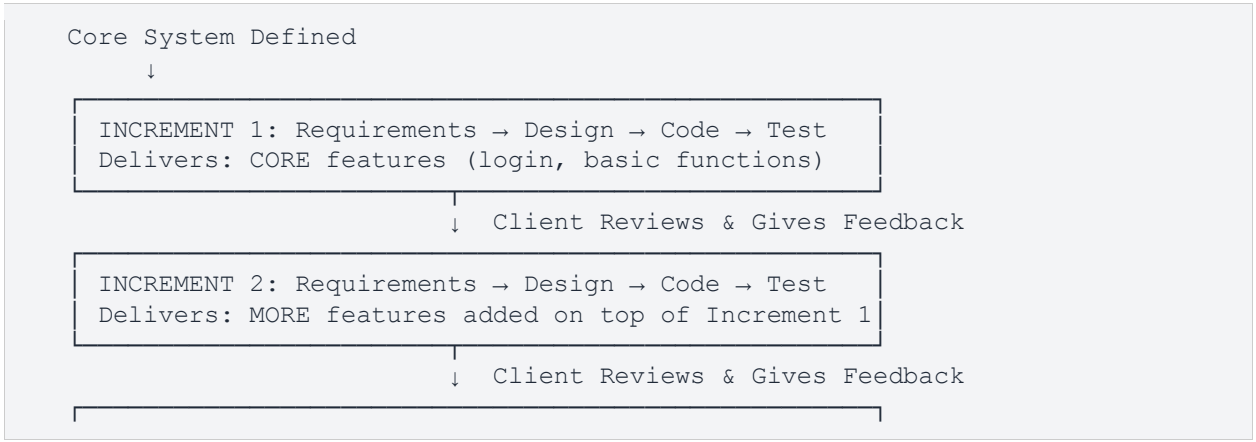
✓When to Use Rapid Prototyping:

Requirements are unclear or hard to define upfront
Client needs to SEE something before they can articulate what they want
UI/UX focused applications — mobile apps, websites, dashboards
Innovative or first-of-a-kind products with no existing reference
Example: A startup building a new food delivery app needs to test the UI with real users before full development

④ Incremental Model

What is it?

The Incremental Model divides the system into small, fully functional pieces called INCREMENTS. Each increment adds more features on top of the previous delivery. Each increment goes through its own mini-waterfall of requirements, design, code, and test.



```
| INCREMENT 3: Requirements → Design → Code → Test    |
| Delivers: COMPLETE system                           |
└─────────────────────────────────────────────────────┘

Example — E-Commerce Site:
Increment 1: User login + product browsing
Increment 2: Shopping cart + checkout + payments
Increment 3: Product reviews + recommendations + analytics
```

✓✓ Advantages	✗ Disadvantages
Working software delivered EARLY and regularly	Requires good planning to define increment boundaries cleanly
Client sees real progress throughout development	Later increments may need to rework earlier code
Easier to manage risk — problems found increment by increment	Total project cost can be hard to predict upfront
Changes can be incorporated in later increments	Needs active client participation for feedback
Core high-priority features delivered first	Architecture must be designed carefully to accommodate future increments

✓✓When to Use Incremental:

Large systems where early delivery of core features is important
Projects where requirements are mostly known but may evolve slightly
When the client wants to start using the system before it is fully complete
Long-term projects where the team wants to reduce risk progressively
Example: Building a hospital management system — deliver patient registration first, then billing, then pharmacy management

⑤ Spiral Model

What is it?

The Spiral Model combines elements of Waterfall AND Rapid Prototyping. Development proceeds in SPIRALS — each loop representing one full phase. The critical difference: RISK ANALYSIS is performed at every single spiral loop before proceeding. It is the best model for managing risk in large, complex projects.

```
Each Spiral Loop has 4 Quadrants:

        Planning   |   Risk Analysis
                   ──────────────────
       Engineering | Evaluation
```

```
QUADRANT 1 — Planning:
  → Define objectives for this spiral
  → Identify constraints and alternatives

QUADRANT 2 — Risk Analysis:
  → Identify potential risks
  → Evaluate alternatives to reduce risk
  → Build prototype if needed to validate

QUADRANT 3 — Engineering:
  → Design, code, and test for this iteration

QUADRANT 4 — Evaluation:
  → Customer reviews this spiral's output
  → Plan the next spiral based on feedback
  → Spiral outward → next loop begins

Spiral 1: Concept exploration
Spiral 2: Requirements + prototype
Spiral 3: Design + detailed prototype
Spiral 4: Full development + testing
Spiral 5: Final delivery
```

✅ Advantages	❌ Disadvantages
Risk is identified and managed at EVERY step	Very expensive — constant risk analysis adds significant cost
Best model for high-risk, complex, large projects	Requires expertise in risk assessment (specialist needed)
Combines flexibility (prototyping) with structure	Not suitable for small or simple projects
Customer is involved throughout all spirals	Can run indefinitely without strict management
Can incorporate changes at each new spiral	Difficult to define clear end point without strong management

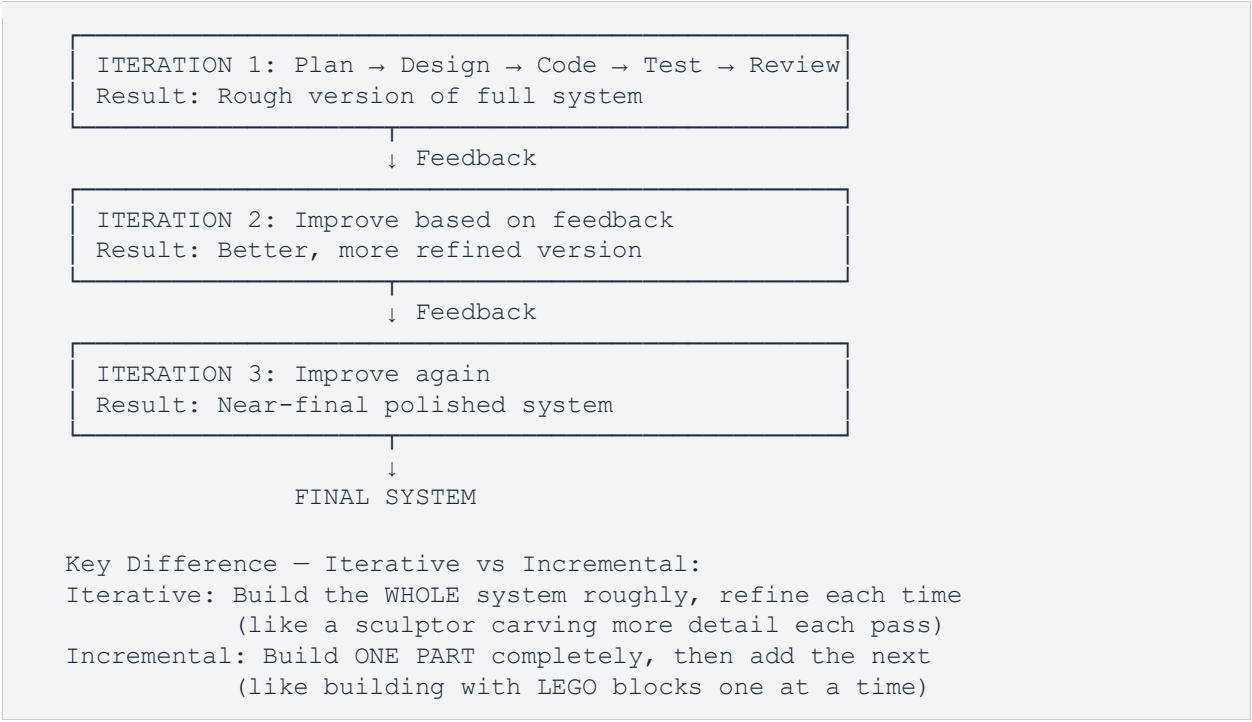
✅When to Use Spiral:

Large, complex projects with significant uncertainty and risk
Government defense or aerospace systems (high stakes)
Projects where technology is new or experimental
Long-term projects (3+ years) where requirements will definitely evolve
When the cost of failure is extremely high
Example: NASA flight control software, national-level banking infrastructure, military command systems

⑥ Iterative Model

What is it?

The Iterative Model builds and improves the system through REPEATED CYCLES called iterations. In each iteration, the team goes through all phases (plan, design, code, test) and produces a working version. Each version is better than the last based on feedback.



✓ Advantages	✗ Disadvantages
Working software available very early	Requires strong project management
Easy to adapt to changing requirements	Total cost/timeline hard to predict upfront
Bugs and issues found much earlier = cheaper to fix	Poor early architecture can break later iterations
Team learns and improves with each iteration	Requires active client participation throughout
High-risk features can be tackled first	Not ideal for very short, simple projects

✓When to Use Iterative:

Large, complex systems where requirements are not fully known upfront
Projects where early feedback is critical to success
Agile/Scrum projects — sprints are essentially iterations
When risk reduction through early working software is a priority
Example: Mobile apps, web platforms, game development — release version 1.0, improve based on user reviews

Process Model Comparison Summary

Model	Flexibility	Client Involvement	Risk Handling	Cost	Best For
Waterfall	Very Low	Start only	Low	Low	Fixed, clear requirements
V-Model	Low	Start + End	Medium	Medium	Safety-critical systems
Rapid Proto.	High	Throughout	High	Medium	Unclear requirements, UI-heavy
Incremental	High	Throughout	Medium	Medium	Large systems, early delivery
Spiral	Very High	Throughout	Very High	Very High	High-risk, complex projects
Iterative	High	Throughout	High	Medium	Evolving requirements, Agile

SECTION 4: UML & USE CASE MODELING

4.1 What is UML?

UML (Unified Modeling Language) is a standardized VISUAL language used to model, design, and document software systems. It gives developers, designers, and clients a COMMON LANGUAGE to communicate about software.

Think of UML as a blueprint language — just like architects use blueprints before building a house, software engineers use UML before building a system.

4.2 Use Case — The Core of UML Behavioral Modeling

A Use Case represents a complete interaction between a USER and the SYSTEM to achieve a specific goal. It is the direct mapping of a Functional Requirement.

4 Elements of a Use Case	Explanation
Actor	WHO interacts with the system
Goal	WHAT they want to achieve
Interaction	HOW they interact with the system
Result	WHAT happens after — the outcome

Key Rule: A task the software does INTERNALLY (not triggered by a user) is NOT a use case. Example: 'Auto-backup database at midnight' — no actor triggers it, so it is NOT a use case.

4.3 Actors — Primary vs Secondary

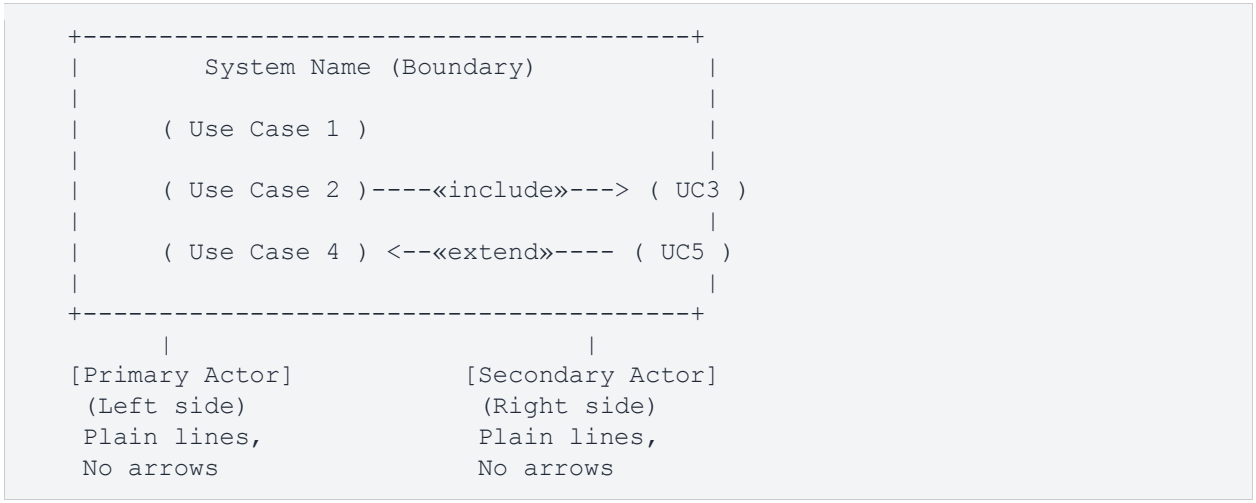
	Primary Actor	Secondary Actor
Definition	Software is made FOR them — they are the intended user	Software is NOT made for them — they react or respond
Initiates UC?	YES — they start the interaction	NO — they respond to system requests
Position in diagram	LEFT side of system boundary	RIGHT side of system boundary
Usually human?	YES — mostly human users	Often NON-HUMAN (external systems)
Examples	Student, Patient, Customer, Driver	SMS System, Payment Gateway, Email Service, GPS

 Critical Rules: (1) A DEVELOPER can NEVER be an actor — they build the system. (2) Admin is usually SECONDARY — the system is not built for admin. (3) Both Driver AND Passenger can be PRIMARY if the app serves both. (4) Actors are ALWAYS outside the system boundary.

4.4 Use Case Diagram — Drawing Rules

Your teacher's specific rules (these will be marked):

- Actor = Stick figure with name written BELOW
- System Boundary = Rectangle with system name inside at the top
- Use Cases = OVALS — always INSIDE the system boundary rectangle
- Actor-to-UseCase lines = PLAIN SOLID LINES — NO ARROWS
- Include/Extend = DASHED LINES with «include» or «extend» label above the line



4.5 «include» vs «extend» — Most Critical Distinction

	«include» (Mandatory)	«extend» (Optional)
Nature	MANDATORY — must always happen	OPTIONAL — may or may not happen
Base UC without it?	✗Cannot function without it	✓Works perfectly without it
Happens every time?	YES — always, no exceptions	NO — only under specific conditions
Arrow direction	Base UC ——→ Included UC	Extending UC ——→ Base UC
Actor connects to included UC?	NO — actor only connects to base	NO — actor only connects to base

Example	Pay Bill «include» Verify Payment	Pay Bill «extend» Print Receipt

Banking App — Full Example:

Withdraw Cash ——«include»——→ Authenticate User (Authentication ALWAYS required)
Transfer Funds ——«include»——→ Authenticate User (Authentication ALWAYS required)
Transfer Funds ——«extend»——→ Generate OTP (Only if amount > Rs. 50,000)
Withdraw Cash ——«extend»——→ Print Receipt (Optional — customer may not want receipt)
Check Balance ——«extend»——→ Print Receipt (Optional)

4.6 Use Case Specification Table

When a diagram is not detailed enough, you write a Use Case Specification — a structured table describing every aspect of the use case:

Field	What to Write & Rules
UC#	Unique number: UC01, UC02, UC03...
Use Case Name	Short verb-noun format: 'Borrow Book', 'Pay Fee', 'Register Account'
Actor(s)	Who initiates this use case (Primary Actor)
Precondition	What MUST be true BEFORE this UC can start. Example: 'UC01 must be completed before UC03 can begin'
Basic Flow	Step-by-step in POSITIVE SENSE — written as if it already succeeded successfully
Alternate Flow	What happens when the basic flow FAILS or goes wrong at each step
Special Req (NFR)	Quality requirement for this use case. Example: 'System must respond within 2 seconds'
Post Condition	The state of the system AFTER successful completion. Example: 'Book marked as borrowed, due date set'
Extension Point	The optional «extend» use case, if any. Example: 'Pay Fine «extend» — triggered if overdue books exist'

Example — UC03: Borrow Book

UC#: UC03
Use Case Name: Borrow Book
Actor(s): Student (Primary)
Precondition: Student must be a registered member (UC01 must be completed first)
Basic Flow: 1. Student searches for book 2. System checks availability 3. Membership verified «include» 4. Book is issued
Alternate Flow: 2a. Book not available → system shows 'unavailable' | 3a. Membership invalid → borrowing denied
Special Req (NFR): System must respond in under 2 seconds

Post Condition: Book marked as borrowed; due-return date set in system
Extension Point: Pay Fine «extend» — triggered if student has overdue books

4.7 Complex Use Case Diagrams

Large systems can have 30–50 use cases, making a single diagram unreadable. Two breakdown approaches:

- By actors/roles — draw a separate diagram for each actor group (e.g. Student diagram, Admin diagram)
- By common functionalities — group related use cases into sub-diagrams

Only use breakdown when the diagram is NOT readable. Don't break it up unnecessarily — over-fragmenting makes it harder to understand the full picture.

SECTION 5: DOMAIN MODEL

5.1 What is a Domain Model?

A Domain Model is a visual representation of real-world CONCEPTS in a specific business domain — showing what EXISTS in that world, their attributes, and how they relate to each other.

Domain examples: healthcare (patients, doctors, appointments), e-commerce (products, orders, customers), education (students, courses, grades).

Critical Rule: Domain Model does NOT fall under UML. It is mainly used in Domain Driven Development (DDD). It shows WHAT exists — NOT HOW it is implemented.

5.2 Domain Model vs Use Case Diagram

	Domain Model	Use Case Diagram
Shows	Real-world concepts & their relationships	User-system interactions and goals
Focus	WHAT exists in the business domain	WHO does WHAT with the system
Is it UML?	NO — not a standard UML diagram	YES — standard UML behavioral diagram
Contains	Classes, attributes, associations	Actors, use cases, include/extend
Has methods?	NO — no methods, no code	N/A
Uses	Business vocabulary only	System/functional terminology

5.3 Domain Model Rules

- Shows WHAT exists — not HOW it is implemented
- Uses BUSINESS vocabulary — terms the client uses, not technical terms
- NO methods or functions — calculateTotal() does NOT belong here
- NO data types written next to attributes
- Plain lines only — NO arrows between classes
- Multiplicity IS shown on association lines

✗Common Mistake: Adding calculateTotal() or processPayment() to a domain model class. These are IMPLEMENTATION details — they belong in design-level class diagrams, NOT in a domain model.

5.4 Attributes vs Classes

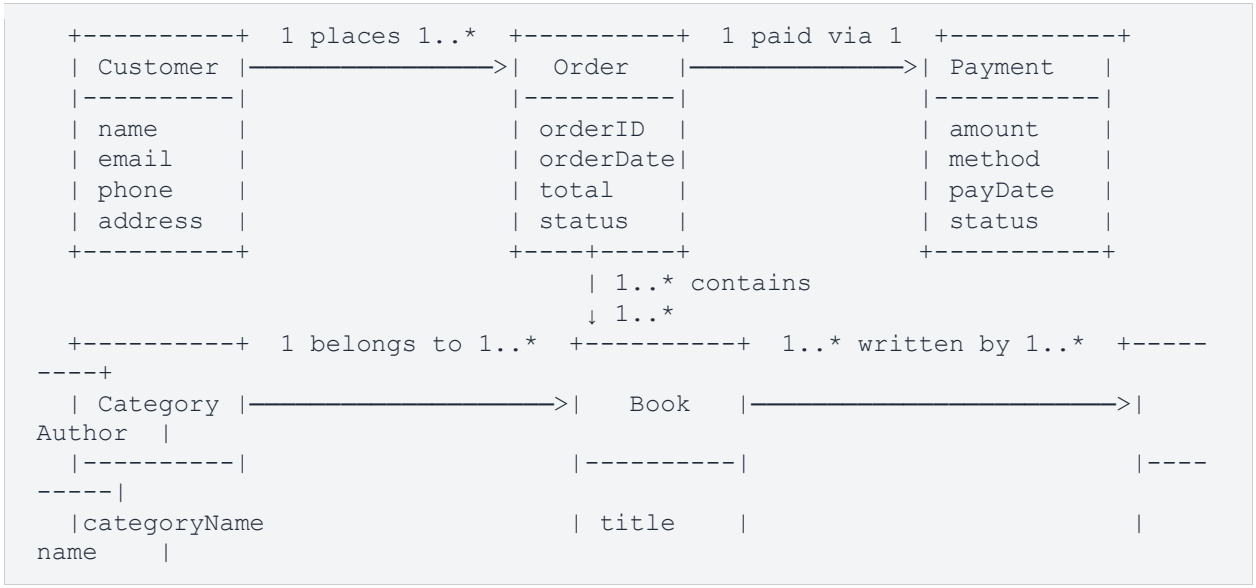
	Class	Attribute
Definition	Has its own identity, properties, behaviours	A simple data value with no independent existence
Can it exist alone?	YES — independently meaningful	NO — only exists as part of a class
Example	'Address' has street, city, postcode → Class	'name' is just a text value → Attribute
Rule of thumb	If in doubt, make it an ATTRIBUTE first	Easier to promote to class later if needed

5.5 Associations & Multiplicity

Associations are relationships between two conceptual classes shown as a plain line. Multiplicity tells you how many instances of one class relate to instances of another.

Notation	Meaning & Example
1	Exactly one. 'An order is paid via exactly 1 payment.'
0..1	Zero or one (optional). 'A customer may have 0 or 1 loyalty card.'
1..*	One or more. 'A customer places 1 or more orders.'
0..* or *	Zero or more. 'A product may have 0 or more reviews.'
4..*	At least four, no upper limit. 'A team has at least 4 members.'
1..3	Between one and three exactly.

Online Bookstore Domain Model example:



```
  |description                    | price    |                              |
|nationality
  +----------+                    | ISBN     |                          | bio
|
                                  +----------+                          +----
-----+

  Rules: Plain lines (no arrows), multiplicity on both ends, NO methods.
```

SECTION 6: COMMON EXAM TRAPS & QUICK REFERENCE

6.1 Common Exam Traps & How to Avoid Them

Trap 1: Developer as Actor

Question: A diagram shows 'Developer' connected to 'Deploy System'. Is this correct?

ANSWER: ✗NO — INCORRECT.
A developer can NEVER be an actor. Actors are always end users or external systems.
Developers are STAKEHOLDERS — they build the system, they are not users of it.
'Deploy System' is also not a user-facing use case — it is a technical internal task.
The correct fix: Remove Developer from the diagram entirely.

Trap 2: NFR classified as FR

Question: Client says 'I want the app to be fast, reliable, and look modern.' Developer treats all as FR. Correct?

ANSWER: ✗NO — Developer is WRONG.
Fast = NFR (Performance) — it describes HOW WELL the system works
Reliable = NFR (Reliability) — a quality attribute
Look modern = NFR (Usability/Aesthetics) — not a feature you code, a quality standard
NONE of these are Functional Requirements.
FR would be specific features: 'login', 'search for products', 'book an appointment'.

Trap 3: calculateTotal() in Domain Model

Question: A teammate adds calculateTotal() to the Order class in the domain model. Correct?

ANSWER: ✗NO — INCORRECT.
Domain Models show WHAT EXISTS in the real world — not HOW it is implemented.
Methods and functions are IMPLEMENTATION details.
calculateTotal() belongs in a design-level CLASS DIAGRAM, not in a domain model.
Domain model classes have ATTRIBUTES only — no methods.

Trap 4: Overdue check — Precondition or «include»?

Question: 'If a student has an overdue book, they cannot borrow.' Is this a precondition or «include»?

ANSWER: PRECONDITION — not «include».
A precondition is a GATE condition that must be TRUE before the use case even BEGINS.
It is listed in the UC Specification table, not drawn as a separate use case flow.
«include» would imply 'Check Overdue Book' is a whole use case that runs inside 'Borrow Book' — that is incorrect.
The correct entry: Precondition: 'Student must have no overdue books.'

Trap 5: Admin classification

Question: Is Admin a Primary or Secondary actor?

ANSWER: Usually SECONDARY.
The system is not BUILT for Admin — Admin just manages/maintains it.
The system is built for the END USERS (customer, student, patient).
However: Always justify based on the specific case — if a system IS built specifically to serve admin operations, they could be primary.
Most common exam answer: Admin = Secondary.

Trap 6: Goldplating vs Scope Creep confusion

Key distinction:

GOLDPLATING = DEVELOPER adds unwanted extras without client approval
SCOPE CREEP = CLIENT keeps adding new requirements mid-project
Both result in: cost overrun, delays, chaos
Responsibility: RE must control BOTH — reject goldplating from developers, manage new requests from clients

6.2 Master Cheat Sheet

Term	Definition / Key Point — Remember This
Software Engineering	Systematic, disciplined, quantifiable approach — PREVENTS CHAOS
SDLC	Requirements → Architecture → Code → Testing → Deployment → Maintenance
Stakeholder	ANYONE affected by or affecting the software (includes developers)
Goldplating	Developer adds unwanted extras → cost, chaos, delays
Scope Creep	Project target drifts from original agreed scope — client adds more and more
70% Rule	70% of software failures = wrong work in requirements phase
Needs vs Wants	Needs = real requirements. Wants = nice-to-haves. RE separates them.

FR	WHAT the system DOES — features (login, search, book, cancel)
NFR	HOW WELL it works — quality (speed, security, reliability, usability)
NFR→FR	Security becomes FR if system handles payments
SRS	Software Requirements Specification — the formal written agreement
Use Case	Actor + Goal + Interaction + Result
Primary Actor	Software made FOR them — LEFT side — initiates use cases
Secondary Actor	Software NOT made for them — RIGHT side — often non-human
Developer as Actor	NEVER — always wrong. Developers are stakeholders, not actors.
System Boundary	Rectangle — actors OUTSIDE, use cases INSIDE
Lines (no arrows)	Plain solid lines — YOUR TEACHER: NO arrows on association lines
«include»	MANDATORY — base UC cannot work without included UC
«extend»	OPTIONAL — base UC works fine without it. Condition-based.
Precondition	Must be TRUE before use case STARTS
Postcondition	State of system AFTER use case successfully COMPLETES
Basic Flow	Written POSITIVELY — as if it already succeeded
Alternate Flow	What happens when basic flow FAILS
Domain Model	NOT UML — shows WHAT exists. Business vocab. Attributes & associations only.
Domain Model Rule	NO methods, NO data types, NO arrows. Plain lines + multiplicity only.
Attribute vs Class	Has independent existence = Class. Just a data value = Attribute.
Association	Plain line between two conceptual classes
Multiplicity	1 = exactly one | 1..* = one or more | 0..* = zero or more | 4..* = at least four
Robustness	NFR — ability to RECOVER after failure
Waterfall	Sequential, no going back. Fixed requirements only.
V-Model	Testing planned parallel to development. Safety-critical systems.
Rapid Prototyping	Build rough prototype early for feedback. Unclear requirements.
Incremental	Build in functional pieces. Early delivery. Large systems.
Spiral	Risk analysis every cycle. High-risk, complex, large projects.
Iterative	Repeated improvement cycles. Evolving requirements. Agile.

Good Luck on Your Exam! You've Got This!

— End of Complete Study Guide —